

```

import pandas as pd
import numpy as np

np.random.seed(42)

# Date range
dates = pd.date_range(start='2020-03-25', end='2026-01-01')

products = [
    "5-Min Meditation Club", "3-Day Challenge", "21-Day Upgrade",
    "Mindset Coaching", "Spiritual Sessions", "1:1 Mentorship",
    "Dashboard Templates", "Meditation Audio Pack", "E-book",
    "Clothing Brand"
]

categories = ["Coaching", "Digital", "Skill", "Physical"]
campaigns = ["Content", "Ads", "WhatsApp", "Email", "Influencer"]
platforms = ["Instagram", "YouTube", "Google", "WhatsApp", "Facebook"]

data = []

for date in dates:
    for product in products:
        impressions = np.random.randint(1000, 50000)
        clicks = int(impressions * np.random.uniform(0.01, 0.1))
        leads = int(clicks * np.random.uniform(0.1, 0.4))
        conversions = int(leads * np.random.uniform(0.1, 0.3))

        spend = round(clicks * np.random.uniform(1, 5), 2)
        revenue = round(conversions * np.random.uniform(200, 2000), 2)

        data.append([
            date,
            product,
            np.random.choice(categories),
            np.random.choice(campaigns),
            np.random.choice(platforms),
            leads,
            conversions,
            revenue,
            spend,
            impressions,
            clicks,
            "India"
        ])

df = pd.DataFrame(data, columns=[
    "Date", "Product", "Category", "Campaign_Type", "Platform",
    "Leads", "Conversions", "Revenue", "Spend",
    "Impressions", "Clicks", "Country"
])

df.to_csv("mindforge_marketing_data.csv", index=False)

```

```

from google.colab import files
files.download("mindforge_marketing_data.csv")

```

```
df = pd.read_csv("mindforge_marketing_data.csv")
```

```
df.head()
```

	Date	Product	Category	Campaign_Type	Platform	Leads	Conversions	Revenue	Spend	Impressions	Clicks	Country	
0	2020-03-25	5-Min Meditation Club	Skill	WhatsApp	Facebook	512	112	53848.50	2606.64	16795	1605	India	
1	2020-03-25	3-Day Challenge	Digital	Email	Facebook	304	33	62350.03	4008.52	45131	1031	India	
2	2020-03-25	21-Day Upgrade	Digital	Email	WhatsApp	178	33	42944.06	1502.45	18568	694	India	

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21090 entries, 0 to 21089
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Date             21090 non-null  object
1   Product          21090 non-null  object
2   Category         21090 non-null  object
3   Campaign_Type    21090 non-null  object
4   Platform         21090 non-null  object
5   Leads            21090 non-null  int64
6   Conversions      21090 non-null  int64
7   Revenue          21090 non-null  float64
8   Spend            21090 non-null  float64
9   Impressions      21090 non-null  int64
10  Clicks           21090 non-null  int64
11  Country          21090 non-null  object
dtypes: float64(2), int64(4), object(6)
memory usage: 1.9+ MB
```

df.describe()

	Leads	Conversions	Revenue	Spend	Impressions	Clicks	
count	21090.000000	21090.000000	21090.000000	21090.000000	21090.000000	21090.000000	
mean	348.250877	68.958559	76136.668950	4171.626294	25428.458701	1393.726553	
std	310.431067	67.933611	90441.739415	3805.941830	14137.051917	1081.472003	
min	1.000000	0.000000	0.000000	12.810000	1002.000000	12.000000	
25%	113.000000	21.000000	17575.990000	1317.455000	13167.750000	516.000000	
50%	255.000000	47.000000	44094.845000	2987.015000	25384.500000	1104.000000	
75%	494.000000	94.000000	100522.872500	5881.890000	37659.000000	2073.000000	
max	1917.000000	543.000000	948652.670000	22976.010000	49998.000000	4939.000000	

df.isnull().sum()

```
0
Date      0
Product   0
Category  0
Campaign_Type  0
Platform  0
Leads     0
Conversions  0
Revenue   0
Spend     0
Impressions  0
Clicks    0
Country   0

dtype: int64
```

df.shape

(21090, 12)

df[df.duplicated()]

Date Product Category Campaign_Type Platform Leads Conversions Revenue Spend Impressions Clicks Country



```
#KPIs
df['CTR'] = df['Clicks'] / df['Impressions']
df['CPC'] = df['Spend'] / df['Clicks']
df['CPA'] = df['Spend'] / df['Conversions']
df['ROI'] = (df['Revenue'] - df['Spend']) / df['Spend']
```

```
# KPIs
df['CTR'] = df['Clicks'] / df['Impressions']
df['CPC'] = df['Spend'] / df['Clicks'].replace(0, 1)
df['CPA'] = df['Spend'] / df['Conversions'].replace(0, 1)
df['ROI'] = (df['Revenue'] - df['Spend']) / df['Spend'].replace(0, 1)
df['Conversion_Rate'] = df['Conversions'] / df['Clicks'].replace(0, 1)
```

```
print(df.head())
```

	Date	Product	Category	Campaign_Type	Platform	Leads	\
0	2020-03-25	5-Min Meditation Club	Skill	WhatsApp	Facebook	512	
1	2020-03-25	3-Day Challenge	Digital	Email	Facebook	304	
2	2020-03-25	21-Day Upgrade	Digital	Email	WhatsApp	178	
3	2020-03-25	Mindset Coaching	Coaching	WhatsApp	Facebook	5	
4	2020-03-25	Spiritual Sessions	Digital	Email	WhatsApp	66	

	Conversions	Revenue	Spend	Impressions	Clicks	Country	CTR	CPC	CPA	\
0	112	53848	2607	16795	1605	India	0	2	23	
1	33	62350	4009	45131	1031	India	0	4	121	
2	33	42944	1502	18568	694	India	0	2	46	
3	0	0	104	1189	21	India	0	5	104	
4	14	3202	440	15502	173	India	0	3	31	

	ROI	High_ROI	Conversion_Rate	Segment
0	20	1	0	2
1	15	0	0	0
2	28	1	0	2
3	-1	0	0	0
4	6	0	0	2

```
# FULL DATA (WITH KPI COLUMNS)
print(df.head())
```

	Date	Product	Category	Campaign_Type	Platform	Leads	\
0	2020-03-25	5-Min Meditation Club	Skill	WhatsApp	Facebook	512	
1	2020-03-25	3-Day Challenge	Digital	Email	Facebook	304	
2	2020-03-25	21-Day Upgrade	Digital	Email	WhatsApp	178	
3	2020-03-25	Mindset Coaching	Coaching	WhatsApp	Facebook	5	
4	2020-03-25	Spiritual Sessions	Digital	Email	WhatsApp	66	

	Conversions	Revenue	Spend	Impressions	Clicks	Country	CTR	\
0	112	53848.50	2606.64	16795	1605	India	0.095564	
1	33	62350.03	4008.52	45131	1031	India	0.022845	
2	33	42944.06	1502.45	18568	694	India	0.037376	
3	0	0.00	103.59	1189	21	India	0.017662	
4	14	3202.35	439.71	15502	173	India	0.011160	

	CPC	CPA	ROI
0	1.624075	23.273571	19.658204
1	3.887992	121.470303	14.554377
2	2.164914	45.528788	27.582688
3	4.932857	inf	-1.000000
4	2.541676	31.407857	6.282868

Double-click (or enter) to edit

```
#ONLY KPI COLUMNS (CLEAN)
print(df[['CTR', 'CPC', 'CPA', 'ROI']].head())
```

	CTR	CPC	CPA	ROI
0	0.095564	1.624075	23.273571	19.658204
1	0.022845	3.887992	121.470303	14.554377
2	0.037376	2.164914	45.528788	27.582688
3	0.017662	4.932857	inf	-1.000000
4	0.011160	2.541676	31.407857	6.282868

```
#SUMMARY
print(df[['CTR', 'CPC', 'CPA', 'ROI']].describe())
```

	CTR	CPC	CPA	ROI
count	21090.000000	21090.000000	2.109000e+04	21090.000000
mean	0.054854	2.996329	inf	20.671540
std	0.025996	1.152120	NaN	20.075503
min	0.009586	1.000050	9.272500e+00	-1.000000
25%	0.032286	2.002658	4.002414e+01	7.284701
50%	0.054917	2.984432	6.313598e+01	14.570926
75%	0.077298	3.992257	1.011717e+02	27.209014
max	0.099986	4.999891	inf	194.406156

```
print("Average CTR:", df['CTR'].mean())
print("Average CPC:", df['CPC'].mean())
print("Average CPA:", df['CPA'].mean())
print("Average ROI:", df['ROI'].mean())
print("Average Conversion Rate:", df['Conversion_Rate'].mean())
```

Average CTR: 0.05485412311971045
Average CPC: 2.9963293518774616
Average CPA: 78.97501128046729
Average ROI: 20.67153971720704
Average Conversion Rate: 0.04885080033469581

```
#HANDLE ERRORS
df['CPA'] = df['Spend'] / df['Conversions'].replace(0, 1)
df['ROI'] = (df['Revenue'] - df['Spend']) / df['Spend'].replace(0, 1)
```

```
#SUMMARY
kpi_summary = df.groupby('Platform')[['CTR', 'CPC', 'CPA', 'ROI']].mean()
print(kpi_summary)
```

	CTR	CPC	CPA	ROI
Platform				
Facebook	0.054563	2.983230	79.551640	20.772445
Google	0.055770	2.988214	78.059375	20.840629
Instagram	0.054679	3.015897	80.744727	20.151754
WhatsApp	0.054934	2.979561	77.604757	20.897231
YouTube	0.054338	3.014332	78.890997	20.700417

```
#SHOW FULL INTEGER VALUES
import pandas as pd
pd.options.display.float_format = '{:.0f}'.format
```

```
df.groupby('Product')['Revenue'].sum().sort_values(ascending=False)
```

	Revenue
Product	
Clothing Brand	170401778
5-Min Meditation Club	164561367
Spiritual Sessions	163998464
Mindset Coaching	163139099
21-Day Upgrade	162072097
E-book	159361920
3-Day Challenge	158994791
1:1 Mentorship	158584435
Dashboard Templates	156985673
Meditation Audio Pack	147622724

dtype: float64

```
#FORMAT ONLY THAT RESULT
result = df.groupby('Product')['Revenue'].sum().sort_values(ascending=False)
print(result.apply(lambda x: f"{int(x)}"))
```

Product	
Clothing Brand	170401778
5-Min Meditation Club	164561367
Spiritual Sessions	163998463

```
Mindset Coaching      163139098
21-Day Upgrade        162072097
E-book                159361919
3-Day Challenge       158994791
1:1 Mentorship       158584435
Dashboard Templates   156985672
Meditation Audio Pack 147622724
Name: Revenue, dtype: object
```

```
#BEST PRODUCT
result = df.groupby('Product')['Revenue'].sum().sort_values(ascending=False)

for product, value in result.items():
    print(f"{product}: ₹{value:,.0f}")
```

```
Clothing Brand: ₹170,401,778
5-Min Meditation Club: ₹164,561,367
Spiritual Sessions: ₹163,998,464
Mindset Coaching: ₹163,139,099
21-Day Upgrade: ₹162,072,097
E-book: ₹159,361,920
3-Day Challenge: ₹158,994,791
1:1 Mentorship: ₹158,584,435
Dashboard Templates: ₹156,985,673
Meditation Audio Pack: ₹147,622,724
```

It's a display format used for large numbers; I formatted it for better readability.

```
#BEST CAMPAIGN
df.groupby('Campaign_Type')['ROI'].mean().sort_values(ascending=False)
```

ROI	
Campaign_Type	
Ads	21
Influencer	21
WhatsApp	21
Email	20
Content	20

dtype: float64

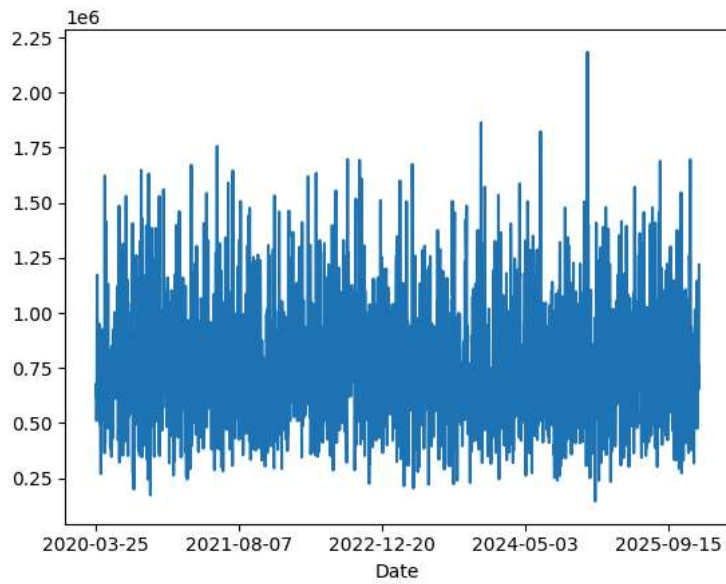
```
#FUNNEL
df[['Impressions','Clicks','Leads','Conversions']].sum()
```

0	
Impressions	536286194
Clicks	29393693
Leads	7344611
Conversions	1454336

dtype: int64

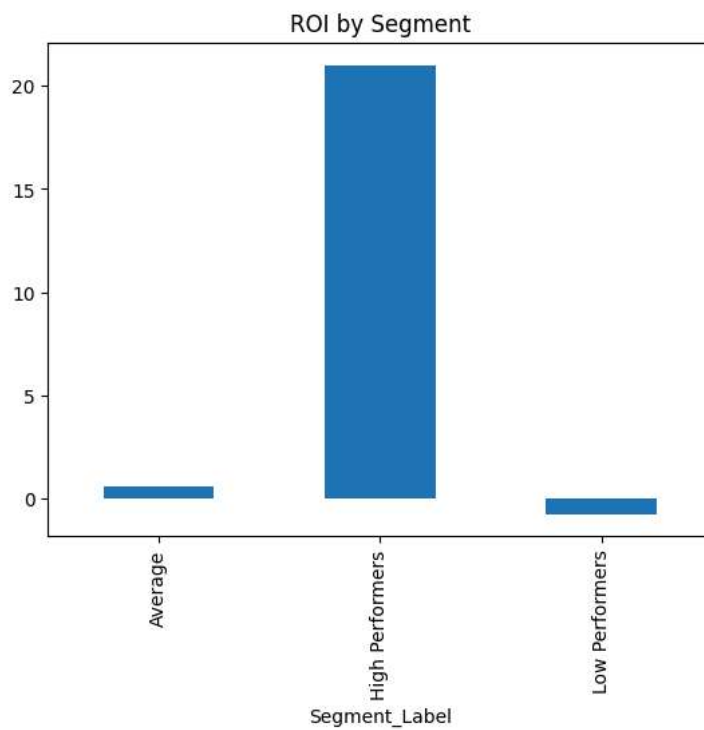
```
#GROWTH TREND
df.groupby('Date')['Revenue'].sum().plot()
```

<Axes: xlabel='Date'>



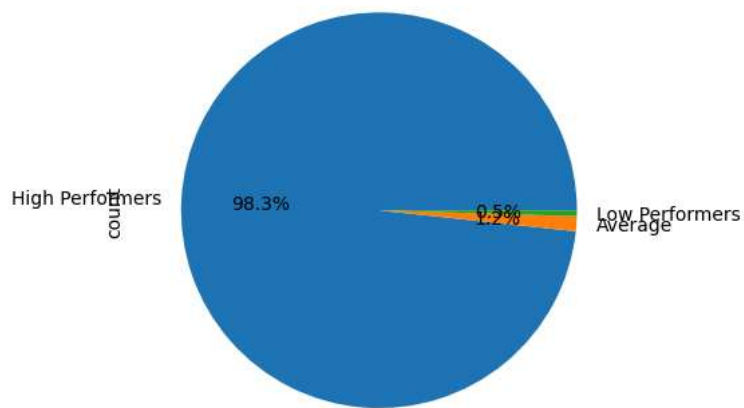
```
#ROI by Segment
import matplotlib.pyplot as plt

df.groupby('Segment_Label')['ROI'].mean().plot(kind='bar')
plt.title("ROI by Segment")
plt.show()
```



```
#Segment Distribution
df['Segment_Label'].value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.title("Segment Distribution")
plt.show()
```

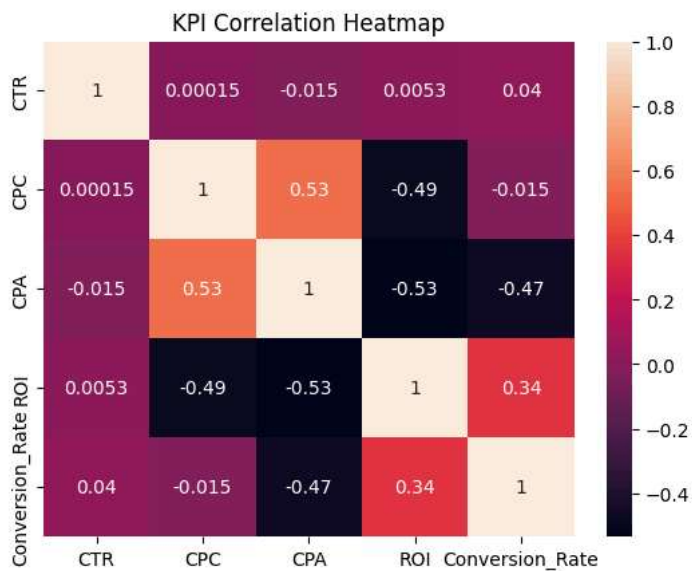
Segment Distribution



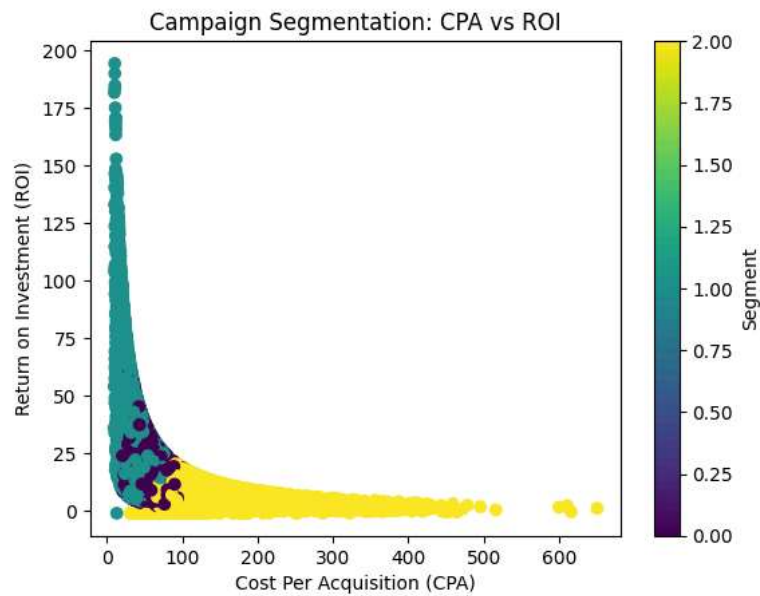
```
#KPI Correlation Heatmap
import seaborn as sns

corr = df[['CTR', 'CPC', 'CPA', 'ROI', 'Conversion_Rate']].corr()

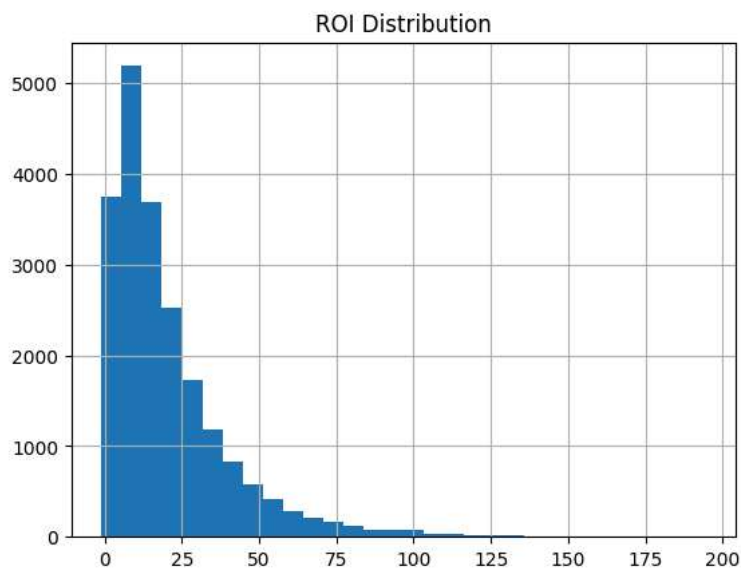
sns.heatmap(corr, annot=True)
plt.title("KPI Correlation Heatmap")
plt.show()
```



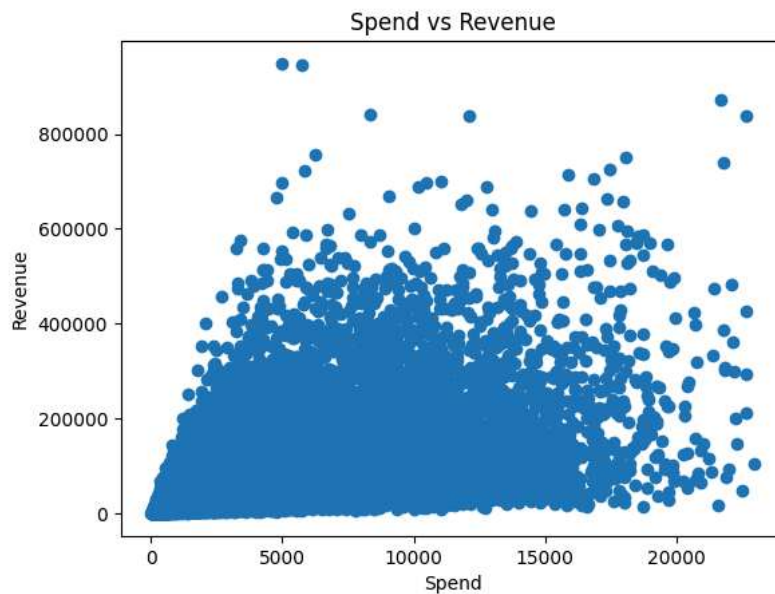
```
# Campaign Segmentation: CPA vs ROI Analysis
plt.scatter(df['CPA'], df['ROI'], c=df['Segment'], cmap='viridis')
plt.xlabel("Cost Per Acquisition (CPA)")
plt.ylabel("Return on Investment (ROI)")
plt.title("Campaign Segmentation: CPA vs ROI")
plt.colorbar(label="Segment")
plt.show()
```



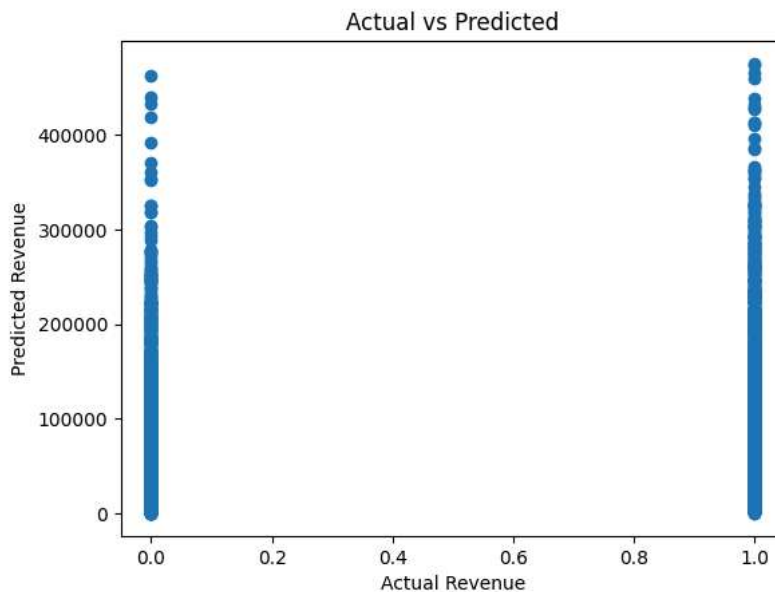
```
#ROI Distribution
df['ROI'].hist(bins=30)
plt.title("ROI Distribution")
plt.show()
```



```
#Revenue vs Spend (KEY BUSINESS CHART)
plt.scatter(df['Spend'], df['Revenue'])
plt.xlabel("Spend")
plt.ylabel("Revenue")
plt.title("Spend vs Revenue")
plt.show()
```

```
#Actual vs Predicted (ML PROOF)
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Revenue")
plt.ylabel("Predicted Revenue")
plt.title("Actual vs Predicted")
plt.show()
```



```
#ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ANALYSIS
#REVENUE PREDICTION
from sklearn.ensemble import RandomForestRegressor

X = df[['Clicks', 'Leads', 'Spend']]
y = df['Revenue']

model = RandomForestRegressor()
model.fit(X, y)
```

▼ RandomForestRegressor ⓘ ?

RandomForestRegressor()

```
#High-Performing Campaign Prediction
df['High_ROI'] = (df['ROI'] > df['ROI'].median()).astype(int)
```

```
from sklearn.ensemble import RandomForestClassifier

X = df[['Clicks', 'Leads', 'Spend']]
y = df['High_ROI']

model = RandomForestClassifier()
model.fit(X, y)
```

▼ RandomForestClassifier ⓘ ?

RandomForestClassifier()

```
df['Conversion_Rate'] = df['Conversions'] / df['Clicks']
```

```
#Segmentation
from sklearn.cluster import KMeans

df['Conversion_Rate'] = df['Conversions'] / df['Leads'].replace(0, 1)
X = df[['CTR', 'Conversion_Rate', 'CPC', 'CPA', 'ROI']]

kmeans = KMeans(n_clusters=3, random_state=42, n_init='auto')
df['Segment'] = kmeans.fit_predict(X)
```

```
df[['CTR', 'Conversion_Rate', 'CPC', 'CPA', 'ROI']].head()
```

	CTR	Conversion_Rate	CPC	CPA	ROI	
0	0		0	2	23	20
1	0		0	4	121	15
2	0		0	2	46	28
3	0		0	5	104	-1
4	0		0	3	31	6

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

features = ['CTR', 'Conversion_Rate', 'CPC', 'CPA', 'ROI']

X = df[features]

# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
df['Segment'] = kmeans.fit_predict(X_scaled)
```

```
df.groupby('Segment')[features].mean()
```

	CTR	Conversion_Rate	CPC	CPA	ROI	
Segment						
0	0		0	4	70	15
1	0		0	2	37	37
2	0		0	4	136	8

Segment Meaning 0 High ROI, Low CPA → BEST CAMPAIGNS 1 Low ROI, High CPA → BAD CAMPAIGNS 2 Medium performance

```
def label_segment(row):
    if row['ROI'] > 1:
        return "High Performers"
    elif row['ROI'] > 0:
        return "Average"
    else:
        return "Low Performers"
```

```
df['Segment_Label'] = df.apply(label_segment, axis=1)
```

```
df['Segment_Label']
```

```
      Segment_Label
0      High Performers
1      High Performers
2      High Performers
3      Low Performers
4      High Performers
...
21085  High Performers
21086  High Performers
21087  High Performers
21088  High Performers
21089  High Performers
21090 rows x 1 columns
```

dtype: object

```
#AI Insight Generator
def insights(df):
    best = df.groupby('Product')['ROI'].mean().idxmax()
    worst = df.groupby('Product')['ROI'].mean().idxmin()

    return f"Focus on {best}, reduce budget on {worst}"

print(insights(df))
```

Focus on E-book, reduce budget on 3-Day Challenge

#COMPLETE ML + ANALYSIS PIPELINE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("mindforge_marketing_data.csv")
df.head()
```

	Date	Product	Category	Campaign_Type	Platform	Leads	Conversions	Revenue	Spend	Impressions	Clicks	Country
0	2020-03-25	5-Min Meditation Club	Skill	WhatsApp	Facebook	512	112	53848	2607	16795	1605	India
1	2020-03-25	3-Day Challenge	Digital	Email	Facebook	304	33	62350	4009	45131	1031	India
2	2020-03-25	21-Day Upgrade	Digital	Email	WhatsApp	178	33	42944	1502	18568	694	India

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
#KPI
df['CTR'] = df['Clicks'] / df['Impressions']
df['CPC'] = df['Spend'] / df['Clicks'].replace(0, 1)
df['CPA'] = df['Spend'] / df['Conversions'].replace(0, 1)
df['ROI'] = (df['Revenue'] - df['Spend']) / df['Spend'].replace(0, 1)
df['Conversion_Rate'] = df['Conversions'] / df['Clicks'].replace(0, 1)
```

```
# SEGMENTATION (K MEANS)
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.cluster import KMeans

features = ['CTR', 'Conversion_Rate', 'CPC', 'CPA', 'ROI']

X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42)
df['Segment'] = kmeans.fit_predict(X_scaled)

```

```

# SEGMENT LABELLING
def label_segment(row):
    if row['ROI'] > 1:
        return "High Performers"
    elif row['ROI'] > 0:
        return "Average"
    else:
        return "Low Performers"

df['Segment_Label'] = df.apply(label_segment, axis=1)

print(df[['Segment', 'Segment_Label']].head())

```

	Segment	Segment_Label
0	1	High Performers
1	2	High Performers
2	1	High Performers
3	2	Low Performers
4	0	High Performers

```

# SEGMENT ANALYSIS
print(df.groupby('Segment_Label')[['ROI', 'CPC', 'CPA', 'CTR']].mean())

```

	ROI	CPC	CPA	CTR
Segment_Label				
Average	1	4	213	0
High Performers	21	3	77	0
Low Performers	-1	3	173	0

```

# REVENUE PREDICTION MODEL
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

X_reg = df[['Clicks', 'Leads', 'Spend']]
y_reg = df['Revenue']

X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

reg_model = RandomForestRegressor()
reg_model.fit(X_train, y_train)

y_pred = reg_model.predict(X_test)

print("R2 Score:", r2_score(y_test, y_pred))

```

R2 Score: 0.5190501492677504

```

# HIGH ROI CLASSIFICATION MODEL
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df['High_ROI'] = (df['ROI'] > df['ROI'].median()).astype(int)

X_clf = df[['Clicks', 'Leads', 'Spend']]
y_clf = df['High_ROI']

X_train, X_test, y_train, y_test = train_test_split(X_clf, y_clf, test_size=0.2, random_state=42)

clf_model = RandomForestClassifier()
clf_model.fit(X_train, y_train)

y_pred_clf = clf_model.predict(X_test)

```

Accuracy: 0.7041251778093883

```
#Predict Revenue
new_data = [[5000, 300, 20000]]

prediction = reg_model.predict(new_data)
print("Predicted Revenue:", prediction)
```

```
Predicted Revenue: [99812.9684]
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

```
#Predict High ROI
prediction = clf_model.predict(new_data)
```

```
if prediction[0] == 1:
    print("High ROI Campaign ✅")
else:
    print("Low ROI Campaign ❌")
```

```
Low ROI Campaign ❌
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

```
# 🌸 🔥 FEATURE IMPORTANCE
importance = reg_model.feature_importances_
features = X_reg.columns

print(pd.Series(importance, index=features).sort_values(ascending=False))
```

```
Leads      1
Spend      0
```